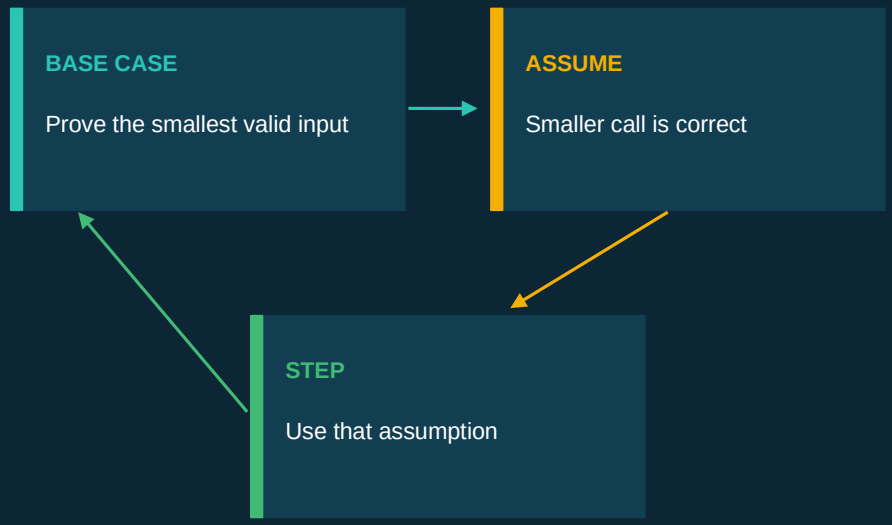
LECTURE 05

Correctness of Recursive Algorithms

Mathematical induction • recursion • base case • inductive step

How do we prove that a recursive call is correct for every valid input?



INDUCTION

a proof structure that mirrors recursion

CDF OUTCOMES

What should you be able to do by the end?

Lecture 5 follows the CDF topic “Correctness of Recursive Algorithms.”

REVIEW

mathematical induction as a proof technique

RELATE

induction to recursive algorithm structures

IDENTIFY

base cases and inductive assumptions

PROVE

correctness across recursive calls

DISTINGUISH

recursive correctness from iterative invariant proofs

COMPARE

recursion and iteration under the same correctness requirements

Required CDF example: prove recursive factorial correctness using mathematical induction.

COURSE MAP

Where Lecture 5 fits

We keep the same correctness goal — but the proof structure changes with the algorithm structure.

L1 What is an algorithm?

L2 How do we design one?

L3 How do we state correctness?

L4 How do we prove loops?

L5 How do we prove recursion?

Lecture 4 used a loop invariant. Lecture 5 uses induction because recursion reduces the problem to smaller copies of itself.

BRIDGE FROM L4

Loop invariant proof vs. recursive induction proof

Same destination: total correctness. Different machinery because the control structure is different.

ITERATIVE ALGORITHM

1. State a loop invariant
2. Initialization
3. Maintenance
4. Termination

The proof follows repeated LOOP iterations.

RECURSIVE ALGORITHM

1. State proposition P(n)
2. Prove the base case
3. Assume smaller case is correct
4. Prove current case
5. Prove recursion terminates

The proof follows repeated RECURSIVE calls.

The proof should mirror the structure of the algorithm.

CORE IDEA

Recursion solves a problem using a smaller version of itself

A recursive algorithm must eventually reach a case it can solve directly.

CURRENT PROBLEM

Solve input of size n

→

REDUCE

Create a smaller instance

→

RECURSIVE CALL

Assume the smaller result is correct

→

COMBINE

Use it to build the answer for n

P(n) depends on P(n − 1)

Correctness asks: if the smaller call returns the right answer, does the current call also return the right answer?

ANATOMY

Every useful recursive algorithm needs three ingredients

Miss one of these and either correctness or termination can fail.

1 BASE CASE

A smallest / simplest valid input that is solved directly — no further recursive call.

2 RECURSIVE CASE

Express the answer for the current input using the answer to a smaller valid input.

3 PROGRESS

Each call must move toward the base case so the recursion cannot continue forever.

Base case supports correctness. Progress supports termination. The recursive case connects the two.

Mathematical induction — the proof pattern

We prove a statement P(n) for every valid integer n.

BASE CASE

Prove P(0) or P(1) directly.

→

INDUCTION HYPOTHESIS

Assume P(k) is true for a smaller k.

→

INDUCTIVE STEP

Using that assumption, prove P(k + 1).

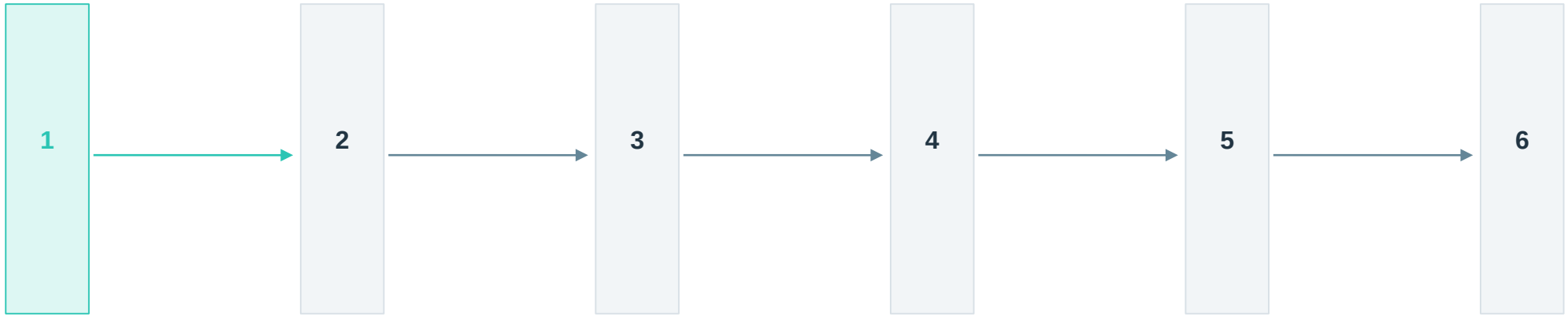
Base + Step ⇒ P(n) holds for every valid n

Induction does not “assume the answer.” It assumes only a smaller case so we can prove the next one.

INTUITION

Think of induction as a line of dominoes

Two facts are enough to establish the whole chain.



BASE CASE

The first domino falls.

INDUCTIVE STEP

If one falls, it knocks down the next.

For recursion: the base case is solved directly; the recursive rule tells us how a correct smaller answer leads to a correct larger answer.

PROOF TEMPLATE

A reusable proof template for recursive algorithms

Use this structure again and again.

1. SPECIFY

State the precondition and postcondition / proposition P(n).

2. BASE CASE

Show the algorithm returns the correct result for the smallest valid input.

3. INDUCTION HYPOTHESIS

Assume the recursive call is correct for a smaller input.

4. INDUCTIVE STEP

Use the algorithm body + hypothesis to prove the current input is correct.

5. TERMINATION

Show each recursive call moves toward a base case.

Partial correctness comes from the induction argument. Termination completes total correctness.

SPECIFICATION

Preconditions and postconditions still come first

A proof is meaningful only after we say what the recursive algorithm promises.

PRECONDITION P

What inputs are valid?

Example for factorial:
n is an integer and n ≥ 0.

POSTCONDITION Q

What must be true when the call returns?

Example:
FACT(n) returns n!.

$$\{ P \} \quad \text{FACT}(n) \quad \{ Q \}$$

The recursive proof shows that every valid call transforms the precondition into the required postcondition.

NOTATION

Mathematical symbols we will use

Read the English meaning first; the notation is only compact shorthand.

Symbol	Read as	Example / English
P(n)	the correctness statement for input n	P(n): FACT(n) returns n!
n!	n factorial	n × (n−1) × … × 1
∀	for every / for all	∀ n ≥ 0 → for every valid n
⇒	implies	P(k) ⇒ P(k+1)
IH	induction hypothesis	assume the smaller recursive call is correct
∈	is an element of	n ∈ ℕ → n is a natural number

Key phrase: the induction hypothesis is about a SMALLER input — not the same input we are trying to prove.

WORKED EXAMPLE 1

Recursive factorial — the required CDF example

Prove that the recursive function returns n! for every integer n ≥ 0.

ALGORITHM

```
FACT(n)
    if n = 0
        return 1
    else
        return n × FACT(n - 1)
```

PRECONDITION

n is an integer and n ≥ 0.

POSTCONDITION

Returned value equals n!.

FACT(n) = n!

English: for every valid n, the function returns the product n × (n−1) × … × 1; by convention 0! = 1.

Question: why is mathematical induction a natural proof for this algorithm?

TRACE

Trace FACT(4) before proving it

A trace shows the recursive structure — but a trace is not a proof for every n.

```
FACT(4)
  4 × FACT(3)
    4 × 3 × FACT(2)
      4 × 3 × 2 × FACT(1)
        4 × 3 × 2 × 1 × FACT(0)
          4 × 3 × 2 × 1 × 1 = 24
```

CALLS GO DOWN

4 → 3 → 2 → 1 → 0

RESULTS COME UP

1 → 1 → 2 → 6 → 24

The proof must explain why this pattern works for every valid n — not only n = 4.

BASE CASE

Factorial proof — Stage 1: Base case

Start with the smallest valid input n = 0.

ALGORITHM DOES

```
if n = 0
    return 1
```

MATHEMATICS SAYS

0! = 1

COMPARE

FACT(0) = 1 = 0!

✓ P(0) is true

Base case complete: the function returns the required value for the smallest valid input.

INDUCTION HYPOTHESIS

Factorial proof — Stage 2: Assume the smaller call is correct

We do not assume FACT(n) is correct. We assume FACT(n−1) is correct.

Assume: FACT(k) = k!

where k is a smaller valid input

WHAT THIS MEANS

If the recursive call is FACT(n−1), we may use:
FACT(n−1) = (n−1)!

WHAT WE STILL MUST PROVE

Using that smaller correct result, show the CURRENT call returns n!.

The induction hypothesis is the bridge that lets the proof “trust” one recursive call.

INDUCTIVE STEP

Factorial proof — Stage 3: Use the hypothesis

Track exactly what the recursive branch returns.

ALGORITHM

FACT(n) returns
n × FACT(n−1)

→

USE IH

FACT(n−1) = (n−1)!

→

SUBSTITUTE

n × (n−1)!

→

FACTORIAL RULE

n × (n−1)! = n!

Therefore FACT(n) = n! ✓

If the smaller recursive call is correct, the current call is correct too.

TERMINATION

Factorial proof — does the recursion actually stop?

Correct arithmetic is not enough; total correctness also needs termination.

WHAT CHANGES?

Each recursive call replaces n by n−1.

WHERE DOES IT GO?

For n ≥ 0, repeated subtraction eventually reaches 0.

WHAT HAPPENS AT 0?

The base case returns directly — no further recursive call.

n → n−1 → n−2 → … → 1 → 0

A useful termination measure is n itself: a non-negative integer that strictly decreases on every recursive call.

TOTAL CORRECTNESS

Factorial — put the whole proof together

Partial correctness + termination gives total correctness.

BASE CASE

FACT(0) returns 1 = 0!
✓

INDUCTIVE STEP

FACT(n−1) correct
⇒ FACT(n) correct
✓

TERMINATION

n decreases to 0
✓

∀ n ≥ 0, FACT(n) returns n! and terminates

Therefore the recursive factorial algorithm is totally correct for its stated input domain.

COMMON ERROR

What if the base case is missing?

The recursive rule may look mathematically reasonable and still never stop.

BROKEN ALGORITHM

```
BAD-FACT(n)
    return n × BAD-FACT(n - 1)
```

TRACE

BAD-FACT(3) → BAD-FACT(2) → BAD-FACT(1) → BAD-FACT(0) → BAD-FACT(−1) → …

WHY IT FAILS

There is no direct-return case. The calls continue forever (or until the runtime fails).

A recursive definition needs a base case that the algorithm can actually reach.

COMMON ERROR

Having a base case is not enough

The recursive call must make progress toward it.

BROKEN ALGORITHM

```
BAD-COUNT(n)
    if n = 0
        return 0
    return 1 + BAD-COUNT(n + 1)
```

BASE CASE EXISTS

n = 0 is present.

BUT PROGRESS IS WRONG

For positive n, the call uses n + 1, moving away from 0.

3 → 4 → 5 → 6 → ...

Termination requires a well-founded measure that moves strictly toward a base case.

CORRECTNESS FAILURE

A recursive algorithm may terminate and still be wrong

Termination and result correctness are separate obligations.

TERMINATES — BUT WRONG

```
WRONG-FACT(n)
    if n = 0
        return 1
    return n + WRONG-FACT(n - 1)
```

ON n = 3

WRONG-FACT(3) = 3 + 2 + 1 + 1 = 7

REQUIRED

3! = 6

DIAGNOSIS

Termination ✓ Postcondition ✗ Total correctness ✗

“It stops” does not imply “it is correct.” Both obligations are required.

WORKED EXAMPLE 2

Recursive SUM(n)

A second example: prove that SUM returns 1 + 2 + … + n.

ALGORITHM

```
SUM(n)
    if n = 0
        return 0
    return n + SUM(n - 1)
```

PRECONDITION

n is an integer and n ≥ 0.

POSTCONDITION

Returned value = n(n+1)/2.

$$SUM(n) = n(n + 1) / 2$$

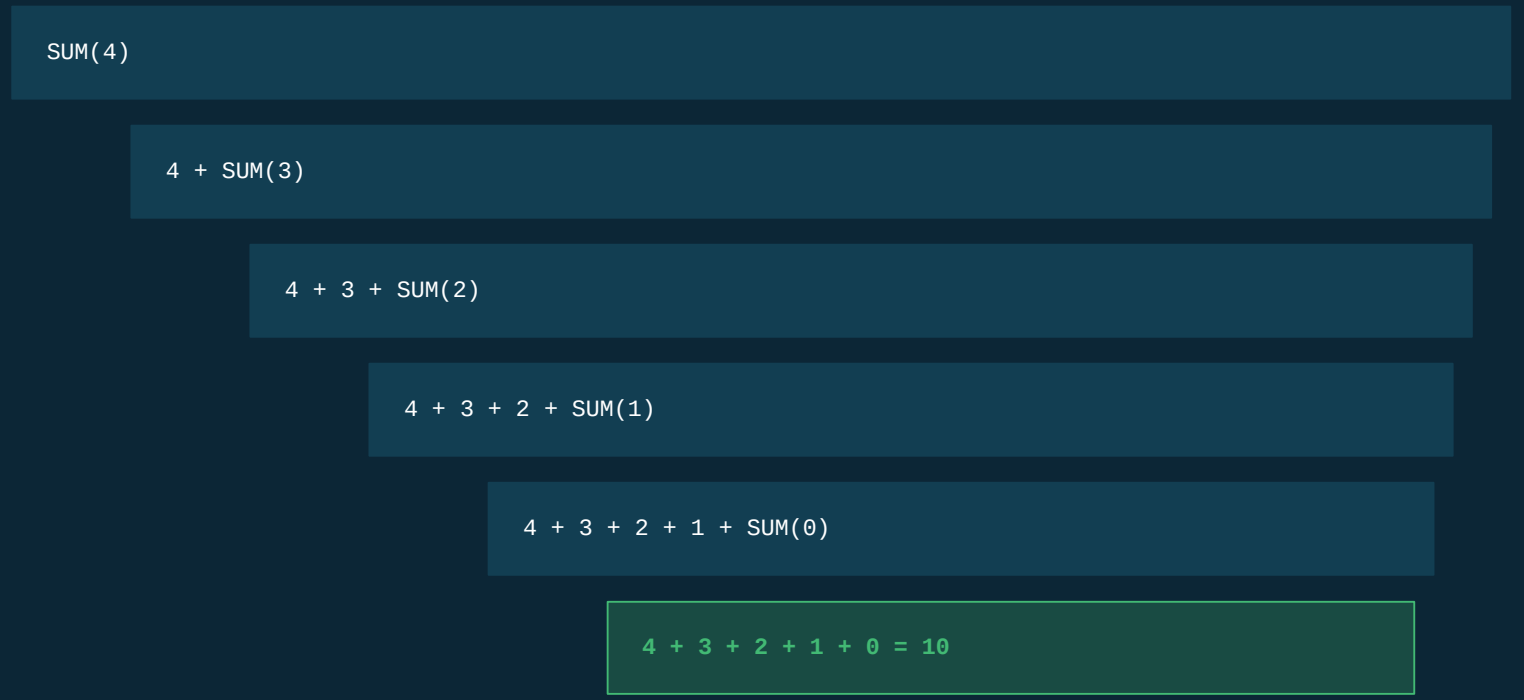
English: the recursive result equals the sum of all integers from 1 through n.

The proof pattern should now feel familiar: base case → hypothesis → step → termination.

TRACE

Trace SUM(4)

Again: useful for understanding, but not a general proof.



SUM(4) = 10 = 4(5)/2

Trace answers “what happened here?” Induction answers “why does it always work?”

SUM(n) — base case and induction hypothesis

Separate what is proved from what is temporarily assumed.

BASE CASE n = 0

SUM(0) returns 0.

Formula gives:
0(0 + 1)/2 = 0

✓ P(0)

INDUCTION HYPOTHESIS

Assume for a smaller k:

SUM(k) = k(k + 1)/2

We will use this to prove SUM(k+1).

The hypothesis gives us the exact value returned by the recursive call.

INDUCTIVE STEP

SUM(n) — the algebra step by step

Use the recursive definition and the induction hypothesis.

START

SUM(k+1)
= (k+1) + SUM(k)

→

USE IH

= (k+1) + k(k+1)/2

→

FACTOR

= (k+1)(k+2)/2

→

MATCH FORMULA

This is formula for n = k+1

SUM(k + 1) = (k + 1)(k + 2) / 2 ✓

The recursive formula plus a correct smaller result produces the correct current result.

TERMINATION

SUM(n) — total correctness

The termination argument is almost identical to factorial.

PARTIAL CORRECTNESS

Base case + induction step prove the returned value is correct if the call returns.

TERMINATION

Each call uses n−1 and valid n is a non-negative integer, so the chain reaches 0.

TOTAL CORRECTNESS

Correct result + guaranteed return for every valid n.

SUM(n) is totally correct for all integers n ≥ 0

Notice how the same termination measure n can support several structurally similar recursive algorithms.

WORKED EXAMPLE 3

Recursive POWER(a,n)

Prove that POWER returns a^n for $n \geq 0$.

ALGORITHM

```
POWER(a, n)
    if n = 0
        return 1
    return a × POWER(a, n - 1)
```

PRECONDITION

n is an integer and $n \geq 0$.

POSTCONDITION

Returned value = a^n.

$$POWER(a,n) = a^n$$

English: multiply a by itself exactly n times; $a^0 = 1$.

Only n controls the recursion depth; a can remain unchanged.

POWER PROOF

POWER(a,n) — see the induction pattern again

Different problem, same proof skeleton.

BASE CASE

n = 0
POWER(a,0) = 1
$a^0 = 1$
✓

HYPOTHESIS

Assume:
POWER(a,k) = a^k

STEP

POWER(a,k+1)
= a × POWER(a,k)
= a × a^k
= a^{k+1}
✓

Termination: n decreases by 1 until 0, so the recursion reaches the base case.

PATTERN

Factorial, SUM, and POWER share the same proof shape

The expressions change; the reasoning structure does not.

Algorithm	Base	Recursive relation	Termination measure
`FACT(n)`	FACT(0)=1	n × FACT(n−1)	n
`SUM(n)`	SUM(0)=0	n + SUM(n−1)	n
`POWER(a,n)`	POWER(a,0)=1	a × POWER(a,n−1)	n

Reusable insight: identify the smallest case, trust one smaller call, show how the current result is built, then prove progress.

COMPARE

Loop invariants vs. induction

Both are structured ways of showing that correctness propagates.

Question	Iterative proof	Recursive proof
Start	Initialization	Base case
Assumption	Invariant true before iteration	Smaller recursive case is correct
Preserve / extend	Maintenance	Inductive step
Finish	Invariant + stopping condition	Base reached + calls return
Termination	Loop progress measure	Recursive progress measure

Different proof technique, same requirement: every valid input must produce the required output and terminate.

Recursive vs. iterative factorial

Implementation style can change — the correctness specification does not.

RECURSIVE

```
FACT-REC(n)
    if n = 0
        return 1
    return n × FACT-REC(n - 1)
```

ITERATIVE

```
FACT-ITER(n)
    result ← 1
    for i ← 1 to n
        result ← result × i
    return result
```

SAME PRE

n is an integer and n ≥ 0

SAME POST

returned value = n!

DIFFERENT PROOF

induction vs. loop invariant

Do not choose an implementation because it “looks clever.” First ensure it satisfies the same correctness contract.

How to evaluate a recursive correctness proof

Put every proof through these checks.

1	**SPECIFICATION**	Are valid inputs and the required result stated precisely?
2	**BASE CASE**	Is the smallest valid case correct?
3	**HYPOTHESIS**	Is the assumption about a strictly smaller valid input?
4	**STEP**	Does the current call become correct if the recursive call is correct?
5	**TERMINATION**	Does every recursive path reach a base case?

A proof is incomplete if any recursive branch lacks correctness reasoning or a termination path.

CLASS PRACTICE

Your turn — identify the proof pieces

For each, state the base case, induction hypothesis, inductive step idea, and termination measure.

A COUNTDOWN

COUNT(n): if n = 0 return; otherwise print n and call COUNT(n−1). What guarantees termination?

B SUM OF SQUARES

SQSUM(n): if n = 0 return 0; otherwise return n^2 + SQSUM(n−1). What postcondition would you prove?

C ARRAY PRODUCT

PROD(A,n): if n = 0 return 1; otherwise return A[n−1] × PROD(A,n−1). What does the recursive call already know?

Start in plain English. Write symbols only after the meaning is clear.

CHECK YOURSELF

Possible answers for the class practice

Equivalent wording is fine if the meaning is precise.

A COUNTDOWN

Base: n=0. Progress/termination measure: n. Each recursive call decreases n by 1 until 0.

B SUM OF SQUARES

Postcondition: SQSUM(n) = $1^2 + 2^2 + \ldots + n^2$. IH: assume SQSUM(k) correctly sums squares through k.

C ARRAY PRODUCT

IH: PROD(A,n−1) equals the product of A[0] through A[n−2]. Multiply by A[n−1] to obtain the product of the first n elements.

The induction hypothesis should describe exactly what the recursive call returns — no more and no less.

RECAP

Six ideas to carry forward

Recursive correctness turns “the recursion seems right” into a structured induction proof.

01 **Specification** — state valid inputs and the required output

02 **Base case** — prove the smallest valid case directly

03 **Induction hypothesis** — assume a smaller recursive call is correct

04 **Inductive step** — use that smaller result to prove the current result

05 **Termination** — show every call moves toward a base case

06 **Same contract** — iteration and recursion must satisfy the same correctness requirements

Next: Analyzing Non-Recursive Algorithms — RAM model, basic operations, and best/worst/average cases.